AIRPORT MANAGEMENT

USING BINARY SEARCH METHOD

TEAM BY :

THUGU .HARSHAVARDHAN REDDY - CDS/2025/2139

BOODATI.JAYANTH-CDS/2025/1814

PROBLEM STATEMENT

- The system must support:
- Add new flights
- Search existing flights
- Update flight details
- Delete flight records
- Display flights in sorted order
- Flight records efficiently is crucial in airports.

REAL-WORLD APPLICATION

- This system can be used in:

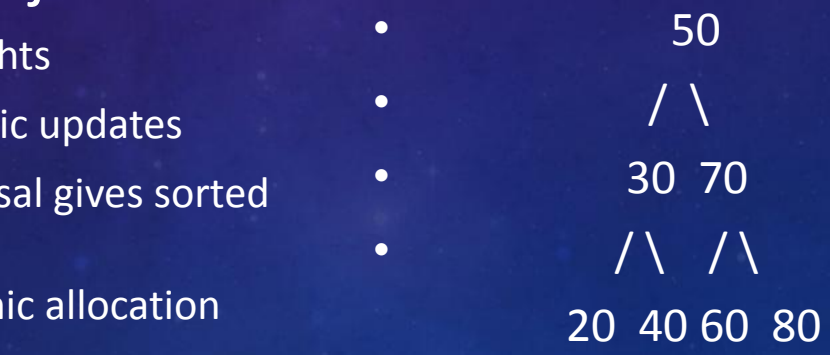
- Airport gate allocation systems
- Flight scheduling systems
- Airline databases
- Railway & bus reservation systems
- ✔ Helps in fast data retrieval
- ✔ Improves efficiency in real-time operations

DATA STRUCTURE USED – BINARY SEARCH TREE (BST)

- What is BST?
- **A Binary Search Tree (BST) is a hierarchical data structure used to store and organize data efficiently.**
- 🧩 Why BST for This Project?
- ⚡ Fast Search ⟶ Quickly locate flights
- 🔄 Efficient Insert/Delete ⟶ Dynamic updates
- 📊 Sorted Output ⟶ In order traversal gives sorted flights
- 🧠 Memory Efficient ⟶ Uses dynamic allocation

Key Advantage

- ✔ Reduces time complexity compared to linear search
- ✔ Ideal for real-time flight management systems



SYSTEM DESIGN

- 1. User Interface
- Menu-based input/output
- Allows user to select operations
- 2. BST Storage
- Stores flight data (Flight No, Gate No)
- Maintains sorted order automatically
- 3. Processing Modules
- Insert Flight
- Delete Flight
- Update Gate
- Search Flight
- Display Flights

- 🔄 Working Flow
- User Input ⟶ Menu Selection ⟶ Function Execution ⟶ BST Operation ⟶ Output Display
- 🎯 Key Features
- ✔ Fast searching and updating
- ✔ Sorted data using in order traversal
- ✔ Dynamic memory allocation
- ✔ Modular structure using functions

CONCEPTS USED

- 1. Functions
- Create Node() Create new node
- insert() Add flight
- search() Find flight
- update() Update gate
- Find Min() Find smallest node
- Delete Node() Delete flight
- display() Show flights

- ✔ Total Functions: 7

- 2. Structure
- struct Node {
- int flight No;
- int gate No;
- struct Node *left, *right;};
- ✔ Used to store flight + gate together
- ✔ Total Structures: 1
- 🔹 3. Loops
- while(1) ⟶ Menu loop
- Recursion ⟶ Tree traversal
- ✔ Used for repeating operations

- 4. Pointers
- Used for:
- Dynamic memory (malloc)
- Linking nodes
- Tree traversal
- ✔ Core of BST implementation
- 5. Variables
- Integer ⟶ flight No, gate No, choice
- Pointer ⟶ root, temp
- ✔ Used for storing and controlling data

- 6. switch-case
- Menu-driven program:
- 1Insert
- 2 Delete
- 3 Update
- 4 Search
- 5 Display
- 6 Exit
- ✔ Better than multiple if-else 🔹

TIME COMPLEXITY

- 7. Binary Search Tree (BST)
- ✔ Advantages :
- Fast searching
- Automatic sorting
- Efficient insert/delete

- Time Complexity
- Operation Complexity
- Insert O(log n)
- Search O(log n)
- Delete O(log n)
- Display O(n)

ALGORITHM

- Insertion

1.Compare the flight number with the root node

2.Insert into left subtree if smaller, right if larger

3.Repeat until correct position is found

- Searching

1.Start from the root node

2.Move left or right based on comparison

3.Stop when the flight is found or node becomes NULL

- Update

1.Search for the flight using Flight Number

2.If found, modify the gate number

3.If not found, display error message

- Traversal (Display)

1.Use In order Traversal (Left → Root → Right)Visits nodes in sorted order

2.Displays all flight records clearly

IMPLEMENTATION

- 🧩 Core Concepts Used
- Structures ⟶ To represent flight records
- Pointers ⟶ For dynamic memory & linking nodes
- Functions ⟶ Modular and reusable code
- Recursion ⟶ Efficient tree traversal
- Switch-case ⟶ Menu-driven user interface
- 🔧 Key Functional Modules
- Create Node() ⟶ Creates a new flight record
- insert() ⟶ Adds a flight into BST
- search() ⟶ Finds a flight by Flight
- Number update() ⟶ Modifies gate number
- Delete Node() ⟶ Removes a flight
- Find Min() ⟶ Finds in order success
- display() ⟶ Shows sorted flight list
- ⚙️ System Features
- ✔ Dynamic memory allocation using malloc()
- ✔ Efficient data handling using BST
- ✔ Sorted output using in order traversal
- ✔ User-friendly menu-driven interface

```
--- ✈Gate Allocation System ---
1. Add Flight
2. Delete Flight
3. Update Gate
4. Search Flight
5. Display Flights
6. Exit
Enter your choice: 1
Enter Flight No and Gate No: 12345
4
```

```
--- ✈Gate Allocation System ---
1. Add Flight
2. Delete Flight
3. Update Gate
4. Search Flight
5. Display Flights
6. Exit
Enter your choice: 3
Enter Flight No to update: 12
```

```
-- ✈Gate Allocation System ---
. Add Flight
. Delete Flight
. Update Gate
. Search Flight
. Display Flights
. Exit
nter your choice: 4
nter Flight No to search: 12345
Flight Found! Gate No: 4
```

```
--- ✈Gate Allocation System ---
1. Add Flight
2. Delete Flight
3. Update Gate
4. Search Flight
5. Display Flights
6. Exit
Enter your choice: 3
Enter Flight No to update: 1234567
Enter New Gate No: 5
❌Flight not found!
```

```
--- ✈Gate Allocation System ---
1. Add Flight
2. Delete Flight
3. Update Gate
4. Search Flight
5. Display Flights
6. Exit
Enter your choice: 1
Enter Flight No and Gate No: 12345
4
```

CONCLUSION

- This project demonstrates the effective use of a Binary Search Tree (BST) for managing airport flight records. ensures:
- ✔ Faster data retrieval compared to traditional methods
- ✔ Automatic sorting using in order traversal
- ✔ Efficient data organization
- The system is:
- ✔ Simple to implement
- ✔ Scalable for moderate datasets
- ✔ Suitable for real-time management systems